

Système Multi-Agents pour la Sélection Intelligente des Candidats en Entreprise

Rapport de Projet Machine Learning
École Centrale Casablanca

Équipe de Développement

Laila AIT BIHI	<code>laila.aitbihi@centrale-casablanca.ma</code>
Chaima AL AYACHI	<code>chaima.alayachi@centrale-casablanca.ma</code>
Salma BOUCHAMA	<code>salma.bouchama@centrale-casablanca.ma</code>
Chaymae DAHHASSI	<code>chaymae.dahhassi@centrale-casablanca.ma</code>

Encadré par : Youssef LAMRANI

25 décembre 2025

Table des matières

I	Introduction	5
0.1	Introduction	6
0.1.1	Contexte et Problématique	6
0.1.2	Objectifs du Projet	6
0.1.3	Innovation et Valeur Ajoutée	6
II	Fondements Théoriques	8
0.2	Introduction aux Fondements Théoriques	9
0.3	Les Systèmes Multi-Agents (SMA)	9
0.3.1	Concepts Fondamentaux	9
0.3.2	Avantages pour le Recrutement	9
0.3.3	Le Framework CrewAI	10
0.4	La Génération Augmentée par la Récupération (RAG)	10
0.4.1	Principe du RAG	10
0.4.2	Fondements Mathématiques	10
0.4.3	Application dans le Projet	11
0.5	Traitement du Langage Naturel (NLP)	11
0.5.1	Named Entity Recognition (NER)	11
0.5.2	Extraction de Compétences	12
0.5.3	Extraction d'Expérience	13
0.6	Modèles de Langage (LLM)	13
0.6.1	Architecture Transformer	13
0.6.2	Mécanisme d'Attention	13
0.6.3	Utilisation dans le Système	13
III	Données, Architecture et Implémentation du Système	15
0.7	Introduction	16
0.8	Données utilisées	16
0.8.1	Description des données	16
0.8.2	Statistiques sur le Dataset	16

0.8.3	Analyse Exploratoire	16
0.9	Architecture Globale et Modulaire	17
0.10	Module de Prétraitement et d'Analyse Exploratoire (EDA)	18
0.11	Module RAG et Base de Données Vectorielle	18
0.12	Le Système Multi-Agents avec CrewAI	18
0.13	Orchestration du Workflow	19
0.14	Synthèse des Technologies	19
IV	Interface Utilisateur et Cas d'Usage	21
0.15	Introduction	22
0.16	Présentation de l'Interface Streamlit	22
0.16.1	Page d'Accueil	22
0.16.2	Page d'Analyse Exploratoire	24
0.16.3	Page d'Évaluation Multi-Agents	26
0.16.4	Page de Résultats	29
0.17	Démonstration d'un Cas d'Usage : Recrutement d'un Data Scientist	32
0.17.1	Input	32
0.17.2	Processus	32
0.17.3	Output	33
V	Conclusion et Perspectives	34
0.18	Synthèse des Points Forts	35
0.19	Pistes d'Amélioration et Travaux Futurs	35
0.19.1	Améliorations Techniques	35
0.19.2	Améliorations Fonctionnelles	36
0.19.3	Améliorations en Explicabilité	36
0.20	Conclusion	37

Table des figures

1	Architecture hiérarchique du système multi-agents	9
2	Pipeline RAG pour la présélection de candidats	11
3	Architecture modulaire du système	17
4	Page d'accueil de l'interface Streamlit	23
5	Présentation de l'architecture multi-agents	23
6	Guide d'utilisation en 3 étapes	24
7	Crédits développeuses	24
8	Interface de chargement des candidats	25
9	Statistiques descriptives des candidats	25
10	Distribution des compétences techniques et soft skills	26
11	option extraction du rapport	26
12	Interface de saisie de la description de poste	27
13	Sélection des candidats à évaluer	27
14	Exemple de description de poste : Data Scientist	28
15	Recherche RAG de candidats pertinents	28
16	Lancement de l'évaluation multi-agents	29
17	Vue d'ensemble des résultats d'évaluation	29
18	Détails de l'évaluation d'un candidat	30
19	Analyse détaillée par agent	30
20	Décision finale avec scores, justifications, points forts et points à améliorer	31
21	Option export des résultats	31
22	Fichier json téléchargé	32

Liste des tableaux

1	Statistiques descriptives du dataset	16
2	Top 3 des compétences techniques	17
3	Pile technologique du système	20

Première partie

Introduction

0.1 Introduction

0.1.1 Contexte et Problématique

Le recrutement moderne fait face à des défis majeurs qui impactent l'efficacité et l'équité du processus de sélection. Les entreprises sont confrontées à :

- **Volume croissant de candidatures** : Des centaines à milliers de CVs par poste à pourvoir
- **Coût temporel élevé** : En moyenne 6-8 minutes par CV pour une première évaluation
- **Subjectivité dans l'évaluation** : Biais cognitifs affectant les décisions de recrutement
- **Manque de transparence** : Difficulté à justifier objectivement les choix de sélection
- **Identification rapide des meilleurs profils** : Nécessité de filtrer efficacement les candidats pertinents

Information

Selon une étude récente, les recruteurs passent en moyenne **6 secondes** sur un CV avant de prendre une décision initiale. Notre système permet une analyse approfondie et objective en quelques minutes seulement.

0.1.2 Objectifs du Projet

Ce projet vise à développer un système intelligent et explicable de sélection de candidats qui :

1. **Automatise l'analyse** des documents de candidature (CVs, lettres de motivation)
2. **Évalue objectivement** les candidats selon des critères multidimensionnels
3. **Fournit des justifications détaillées** et transparentes pour chaque décision
4. **Simule un comité de recrutement** via une architecture multi-agents
5. **Offre une interface web interactive** pour les recruteurs

0.1.3 Innovation et Valeur Ajoutée

Notre solution se distingue par plusieurs innovations clés :

Architecture Multi-Agents Simulation d'un comité d'experts via CrewAI, où chaque agent apporte une expertise spécialisée

Système RAG Recherche sémantique pour une présélection intelligente basée sur la compréhension du contexte

Explicabilité Native Génération automatique de justifications détaillées par LLM pour chaque évaluation

Approche Holistique Évaluation des dimensions technique, comportementale et culturelle simultanément

Interface Moderne Application Streamlit prête à l'emploi avec visualisations interactives

Deuxième partie

Fondements Théoriques

0.2 Introduction aux Fondements Théoriques

Une compréhension solide des concepts théoriques sous-jacents est essentielle pour appréhender les choix d'architecture et de technologie qui structurent ce système. Cette section sert de socle de connaissances, non seulement pour justifier la conception actuelle, mais aussi pour guider les ingénieurs qui seront chargés de la maintenance ou de l'évolution future du projet. Elle présente les trois piliers technologiques sur lesquels repose l'ensemble de la solution.

0.3 Les Systèmes Multi-Agents (SMA)

0.3.1 Concepts Fondamentaux

Un système multi-agents (SMA) est un paradigme informatique où plusieurs agents autonomes et intelligents interagissent dans un environnement commun pour résoudre des problèmes qui dépassent leurs capacités individuelles.

Définition

Un **agent** est une entité autonome capable de :

- Percevoir son environnement
- Prendre des décisions
- Exécuter des actions
- Communiquer avec d'autres agents

0.3.2 Avantages pour le Recrutement

L'avantage de cette approche pour une tâche complexe comme le recrutement est sa capacité à décomposer le problème en sous-tâches spécialisées. Le système simule ainsi un "comité virtuel" où chaque agent évalue les candidats sous un angle différent (technique, humain, cohérence du profil), reflétant la dynamique d'un véritable comité de recrutement.

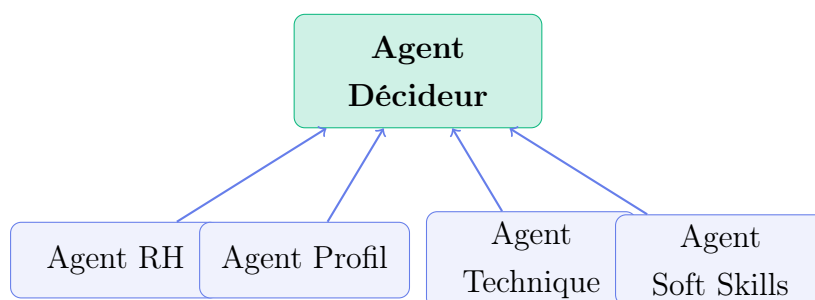


FIGURE 1 – Architecture hiérarchique du système multi-agents

0.3.3 Le Framework CrewAI

Pour l'implémentation de cette logique agentique, le framework **CrewAI** a été choisi pour ses avantages :

- **Définition déclarative des agents** : Rôles, objectifs et interactions définis de manière claire
- **Orchestration de tâches** : Gestion automatique de l'exécution séquentielle ou parallèle
- **Mémoire partagée** : Les agents peuvent accéder aux résultats des autres agents
- **Intégration native avec les LLMs** : Support direct pour Groq, OpenAI, et autres fournisseurs

0.4 La Génération Augmentée par la Récupération (RAG)

0.4.1 Principe du RAG

La génération augmentée par la récupération (RAG) est une technique qui enrichit les capacités des grands modèles de langage (LLM) en leur donnant accès à une base de connaissances externe.

Le RAG combine récupération d'information et génération de texte pour améliorer les LLMs. Le processus comprend trois phases :

1. **Indexation** : Conversion des documents en embeddings vectoriels
2. **Récupération** : Recherche des documents pertinents via similarité sémantique
3. **Augmentation-Génération** : Enrichissement du contexte et génération par LLM

0.4.2 Fondements Mathématiques

Formule probabiliste du RAG :

$$P(y|q) = \sum_{d \in \mathcal{C}} P(y|q, d) \cdot P(d|q) \quad (1)$$

où :

- q est la requête (description de poste)
- y est la réponse générée
- d est un document de la collection $\mathcal{C}$
- $P(d|q)$ est la probabilité de récupération du document d étant donné q
- $P(y|q, d)$ est la probabilité de génération de y étant donné q et d

Embeddings sémantiques : Nous utilisons le modèle `all-MiniLM-L6-v2` (384 dimensions, architecture Transformer 6 couches). La similarité est calculée via distance cosinus :

$$\text{sim}(t_1, t_2) = \frac{\mathbf{e}_1 \cdot \mathbf{e}_2}{\|\mathbf{e}_1\| \|\mathbf{e}_2\|} = \cos(\theta) \quad (2)$$

où $\mathbf{e}_1$ et $\mathbf{e}_2$ sont les vecteurs d'embedding des textes t_1 et t_2 , et θ est l'angle entre ces vecteurs.

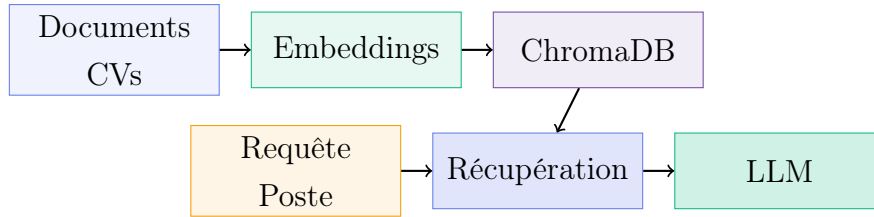


FIGURE 2 – Pipeline RAG pour la présélection de candidats

0.4.3 Application dans le Projet

Dans ce projet, le RAG est appliqué pour effectuer une présélection intelligente : une base de données vectorielle est utilisée pour rechercher et extraire les CVs et documents les plus pertinents par rapport aux exigences formulées dans une description de poste.

Les technologies spécifiques implémentées pour ce module sont :

- **ChromaDB** pour le stockage vectoriel avec index HNSW (Hierarchical Navigable Small World)
- **Embeddings all-MiniLM-L6-v2** pour la conversion des documents textuels en vecteurs numériques

En ancrant le raisonnement des agents sur des extraits de CV pertinents et factuels, le RAG agit comme un garde-fou contre les hallucinations du LLM, garantissant que les évaluations sont fondées sur les données réelles du candidat.

0.5 Traitement du Langage Naturel (NLP)

0.5.1 Named Entity Recognition (NER)

La reconnaissance d'entités nommées (Named Entity Recognition) consiste à identifier et classer automatiquement des entités sémantiques présentes dans un texte. Dans ce projet, nous utilisons la bibliothèque `spaCy` avec le modèle pré-entraîné `fr_core_news_sm`, spécifiquement conçu pour le traitement du langage naturel en français.

Définition

Le modèle `fr_core_news_sm` permet l'extraction des principales catégories d'entités :

- **PERSON** : noms de personnes
- **ORG** : organisations et entreprises
- **LOC** : lieux géographiques
- **DATE** : expressions temporelles

D'un point de vue architectural, le modèle repose sur un réseau de neurones convolutif (CNN) combiné à un parseur transitionnel. L'annotation des entités suit le schéma **BIO** (*Begin-Inside-Outside*), dans lequel chaque token est étiqueté selon sa position au sein d'une entité nommée.

Exemple

Exemple d'annotation BIO :

- **B-PERSON** : Début d'un nom de personne
- **I-PERSON** : Intérieur d'un nom de personne
- **O** : Token hors entité

Pour "Jean Dupont travaille chez Google" :

- Jean : **B-PERSON**
- Dupont : **I-PERSON**
- travaille : **O**
- chez : **O**
- Google : **B-ORG**

En termes de performance, le modèle atteint un **F1-score d'environ 84%** sur des benchmarks de référence en langue française, ce qui témoigne d'un bon compromis entre précision et rappel pour des tâches d'extraction d'information en contexte réel.

0.5.2 Extraction de Compétences

L'extraction de compétences utilise une méthode hybride combinant :

- **Dictionnaire prédéfini** : 30+ compétences techniques, 15+ soft skills
- **Matching case-insensitive** : Recherche insensible à la casse dans le texte
- **Contexte sémantique** : Analyse du contexte d'utilisation des compétences

0.5.3 Extraction d'Expérience

L'extraction d'expérience utilise des expressions régulières pour détecter les patterns comme :

- "expérience de X ans"
- "X années d'expérience"
- "depuis X ans"

0.6 Modèles de Langage (LLM)

0.6.1 Architecture Transformer

Au cœur du système se trouvent les grands modèles de langage (LLM), qui fournissent les capacités de raisonnement et de compréhension du langage.

Ce projet s'appuie sur le modèle **Llama 3.3 70B** via l'API Groq pour bénéficier d'une inférence ultra-rapide (500+ tokens/s) pour alimenter les agents.

Information

Caractéristiques du modèle Llama 3.3 70B :

- **70 milliards** de paramètres
- Fenêtre contextuelle de **8192 tokens**
- Entraînement sur **15 trillions** de tokens
- Architecture **Transformer** avec attention multi-têtes

0.6.2 Mécanisme d'Attention

Le mécanisme d'attention permet au modèle de se concentrer sur les parties pertinentes du contexte. Pour une séquence d'entrée $\mathbf{X} = [x_1, x_2, \dots, x_n]$, l'attention est calculée comme :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3)$$

où Q (Query), K (Key), et V (Value) sont des matrices dérivées de $\mathbf{X}$.

0.6.3 Utilisation dans le Système

Chaque agent utilise le LLM pour :

1. Analyser les informations qui lui sont fournies (fragments de CV, description de poste)

2. Évaluer les candidats selon ses critères spécifiques
3. Générer des justifications textuelles claires et explicables

Troisième partie

Données, Architecture et Implémentation du Système

0.7 Introduction

Cette section constitue le cœur de ce rapport technique. Elle décompose l'architecture globale du système en ses modules principaux, décrivant en détail le rôle de chaque composant, les technologies utilisées et la logique d'interaction qui les unit pour former un pipeline de traitement cohérent et efficace. L'accent est mis sur la modularité et la clarté de la conception, qui sont des facteurs clés pour la maintenabilité et l'évolutivité du projet.

0.8 Données utilisées

0.8.1 Description des données

Notre dataset comprend :

- **Documents candidats** : CVs en PDF/DOCX/TXT (500-1500 mots en moyenne)
- **Descriptions de postes** : Fichiers .txt structurés contenant les exigences du poste

0.8.2 Statistiques sur le Dataset

Sur un échantillon de 10 candidats analysés, nous avons obtenu les statistiques suivantes :

Métrique	Description	Valeur
Mots par CV (moyenne)	Nombre moyen de mots dans les CVs	852
Phrases par CV (moyenne)	Nombre moyen de phrases	38
Compétences techniques (moyenne)	Nombre moyen de compétences techniques identifiées	4.2
Soft skills (moyenne)	Nombre moyen de soft skills identifiées	2.8
Années d'expérience (moyenne)	Expérience professionnelle moyenne	3.4 ans

TABLE 1 – Statistiques descriptives du dataset

0.8.3 Analyse Exploratoire

Distribution des Compétences

Les compétences techniques les plus fréquentes dans notre dataset sont :

Compétence	Fréquence (%)
Python	70%
Machine Learning	50%
SQL	40%

TABLE 2 – Top 3 des compétences techniques

Distribution de l'Expérience

La répartition des candidats selon leur niveau d'expérience :

- Juniors (0-2 ans) : 30%
- Confirmés (2-5 ans) : 50%
- Seniors (5+ ans) : 20%

Corrélations

Une corrélation modérée positive (coefficient de Pearson $r = 0.65$) a été observée entre l'expérience et la longueur du CV, suggérant que les candidats plus expérimentés tendent à avoir des CVs plus détaillés.

0.9 Architecture Globale et Modulaire

Le projet est structuré selon une architecture modulaire, où chaque grande fonctionnalité est encapsulée dans son propre répertoire. Cette organisation favorise la séparation des préoccupations et rend le code plus facile à comprendre et à maintenir.

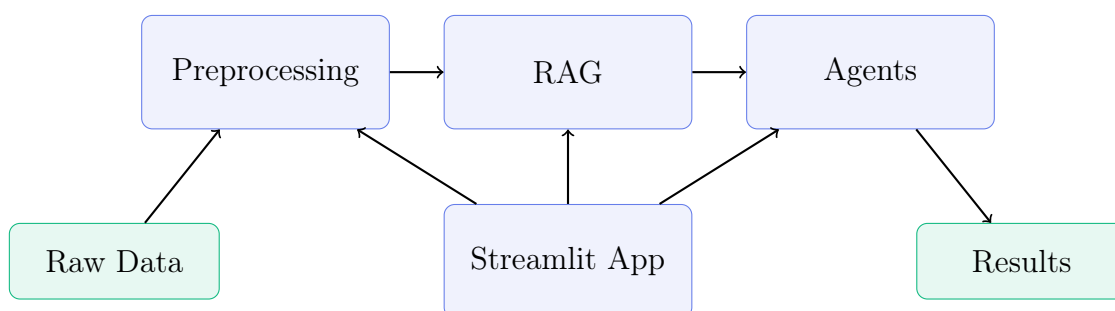


FIGURE 3 – Architecture modulaire du système

Les principaux répertoires, tels que `preprocessing`, `rag`, et `agents`, illustrent cette approche. La gestion de la configuration est centralisée dans un fichier unique (`utils/config.py`), et les dépendances du projet sont explicitement listées dans le fichier `requirements.txt`, conformément aux bonnes pratiques d'ingénierie logicielle.

0.10 Module de Prétraitement et d'Analyse Exploratoire (EDA)

Ce module est la première étape du pipeline et a pour fonction de préparer les données brutes des candidats pour les analyses ultérieures. Il est composé des sous-modules suivants :

`preprocessing/data_loader.py` Gère le chargement des CVs et lettres de motivation à partir de divers formats de fichiers (PDF, DOCX, TXT) et en extrait le contenu textuel brut.

`preprocessing/text_processor.py` Prend en charge le nettoyage et la normalisation du texte. Il utilise la bibliothèque spaCy pour des tâches avancées comme l'extraction d'entités nommées (NER) et l'identification systématique des compétences techniques et interpersonnelles.

`preprocessing/exploratory_analysis.py` Fournit des outils pour calculer des statistiques descriptives sur le corpus de candidats et générer des visualisations (avec matplotlib, seaborn et plotly) pour analyser, par exemple, la distribution des compétences.

0.11 Module RAG et Base de Données Vectorielle

Ce module implémente le système de génération augmentée par la récupération (RAG) pour la présélection des candidats.

`rag/vector_store.py` Gère l'intégration avec la base de données vectorielle ChromaDB. Il utilise les embeddings all-MiniLM-L6-v2 pour convertir le contenu textuel de chaque document candidat en un vecteur numérique dense. Ces vecteurs sont ensuite stockés et indexés à l'aide de l'algorithme HNSW (Hierarchical Navigable Small World), qui offre un compromis efficace entre vitesse de recherche et précision, le rendant particulièrement adapté aux applications interactives à faible latence.

`rag/retriever.py` Orchestre le processus de recherche. En se basant sur les exigences du poste, il interroge la base de données vectorielle pour identifier les candidats les plus pertinents via la similarité cosinus, une métrique qui mesure l'orientation plutôt que la magnitude des vecteurs, la rendant particulièrement efficace pour évaluer la pertinence sémantique indépendamment de la longueur du document.

0.12 Le Système Multi-Agents avec CrewAI

Le cœur du raisonnement du système est géré par une équipe d'agents spécialisés, développés et orchestrés à l'aide du framework CrewAI. Chaque agent a un rôle, des

outils et des objectifs clairement définis :

Agent RH (`hr_agent.py`) Cet agent est le point de départ du processus d'évaluation. Sa responsabilité est d'analyser en profondeur la description de poste fournie par le recruteur pour en extraire les critères essentiels (compétences techniques, niveau d'expérience, qualités humaines, etc.).

Agent Profil (`profile_agent.py`) Il se concentre sur le candidat dans sa globalité. En analysant le CV et la lettre de motivation, il évalue la cohérence du parcours professionnel, la progression de carrière et la pertinence générale du profil par rapport au secteur d'activité.

Agent Technique (`technical_agent.py`) Cet agent est le spécialiste technique de l'équipe. Il évalue spécifiquement les compétences techniques mentionnées par le candidat et les compare rigoureusement aux exigences techniques du poste.

Agent Soft Skills (`soft_skills_agent.py`) Son rôle est d'analyser les aspects humains du candidat. Il recherche des indicateurs de qualités interpersonnelles (communication, travail d'équipe, etc.) et évalue l'adéquation culturelle potentielle avec l'entreprise.

Agent Décideur (`decider_agent.py`) C'est le chef d'orchestre final. Il a la tâche cruciale d'agrégier les scores et les analyses produits par tous les autres agents. Sur cette base, il génère le classement final, rédige des justifications détaillées et explicables pour chaque classement, et compile le rapport final.

0.13 Orchestration du Workflow

La coordination de l'ensemble de ces modules et agents est assurée par le script `candidate_selection_system.py`. Ce module agit comme le *control plane* du système, garantissant un pipeline d'exécution déterministe et reproductible. Il orchestre le flux de données, depuis les candidats bruts jusqu'au rapport final, en s'assurant que chaque étape (prétraitement, récupération, évaluation agentique) est exécutée dans un ordre strict, ce qui est essentiel pour la fiabilité et la traçabilité des résultats.

0.14 Synthèse des Technologies

Le tableau suivant synthétise la pile technologique utilisée pour la construction de ce système :

Domaine	Technologies
Framework Agentique	CrewAI 0.22.0
NLP	spaCy 3.7.2, Sentence-Transformers 2.2.2
RAG	ChromaDB 0.4.24, all-MiniLM-L6-v2 embeddings
LLM	Groq API 0.4.2, Llama 3.3 70B
Interface	Streamlit 1.29.0
Visualisation	Plotly 5.18.0, Matplotlib 3.8.2, Seaborn 0.13.0
Traitement de Données	Pandas 2.1.3, NumPy 1.26.2

TABLE 3 – Pile technologique du système

Quatrième partie

Interface Utilisateur et Cas d'Usage

0.15 Introduction

Cette section est dédiée à la démonstration pratique du système. Elle décrit l'interface utilisateur développée avec Streamlit, qui sert de portail d'interaction pour le recruteur, et illustre le workflow complet du système à travers un scénario de recrutement réaliste, de la saisie des besoins à l'obtention des résultats finaux.

0.16 Présentation de l'Interface Streamlit

L'application web, encapsulée dans le fichier `app.py`, constitue le point d'entrée unique et convivial pour l'utilisateur. Elle est conçue pour être intuitive et guider le recruteur à travers les différentes étapes du processus. L'interface est structurée en plusieurs pages ou onglets, chacun ayant une fonction précise :

Page d'accueil Fournit une présentation générale du système, de ses objectifs et de son fonctionnement. Elle présente l'architecture multi-agents et guide l'utilisateur dans l'utilisation de la plateforme.

Page d'analyse exploratoire Cette page offre au recruteur une vue d'ensemble immédiate de son vivier de talents, lui permettant d'identifier les tendances et la distribution des compétences clés avant même de lancer une évaluation individuelle.

Page d'évaluation multi-agents C'est ici que le processus d'évaluation est initié. L'utilisateur peut saisir ou coller une description de poste complète, puis lancer l'analyse par le système multi-agents.

Page de résultats Plus qu'un simple classement, cette page est un véritable outil d'aide à la décision, fournissant des arguments qualitatifs et quantitatifs pour chaque candidat afin de faciliter et d'objectiver les débriefings de recrutement. Elle présente des visualisations interactives créées avec Plotly pour explorer les résultats.

0.16.1 Page d'Accueil

La page d'accueil présente le système de manière claire et professionnelle. Elle inclut :

- Une présentation visuelle de l'architecture multi-agents
- Un guide d'utilisation en 3 étapes
- Des informations sur la structure des données
- Les crédits de l'équipe de développement

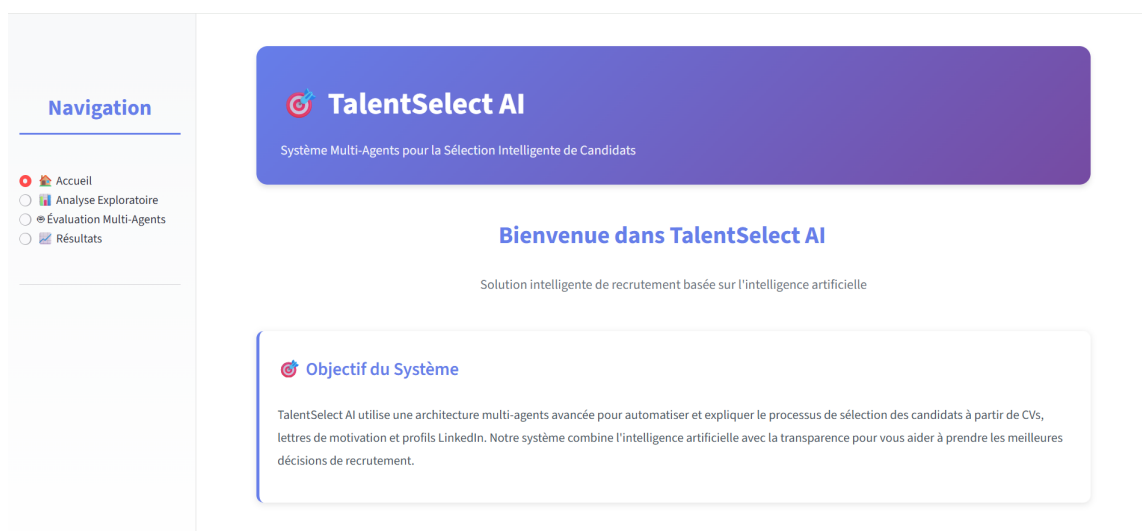


FIGURE 4 – Page d'accueil de l'interface Streamlit

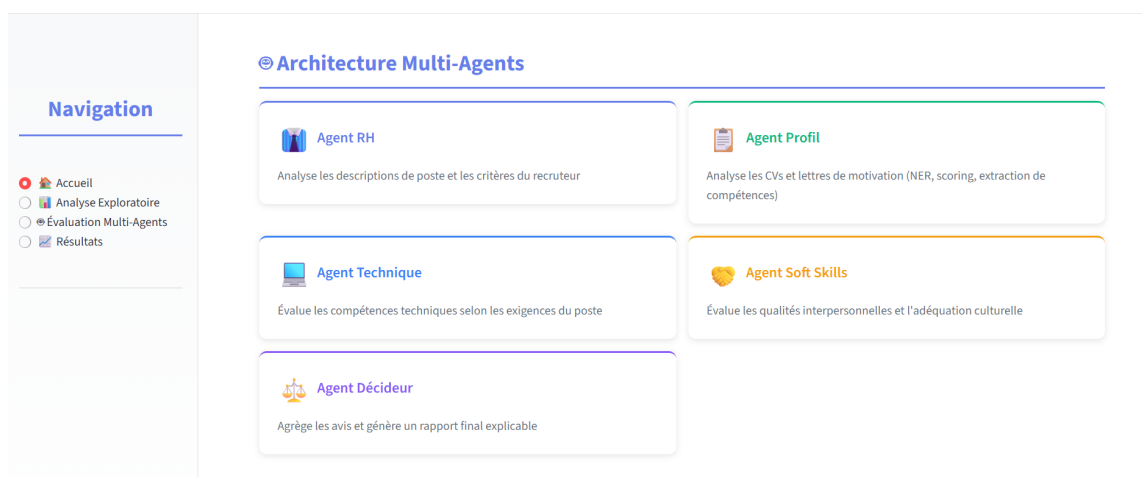


FIGURE 5 – Présentation de l'architecture multi-agents

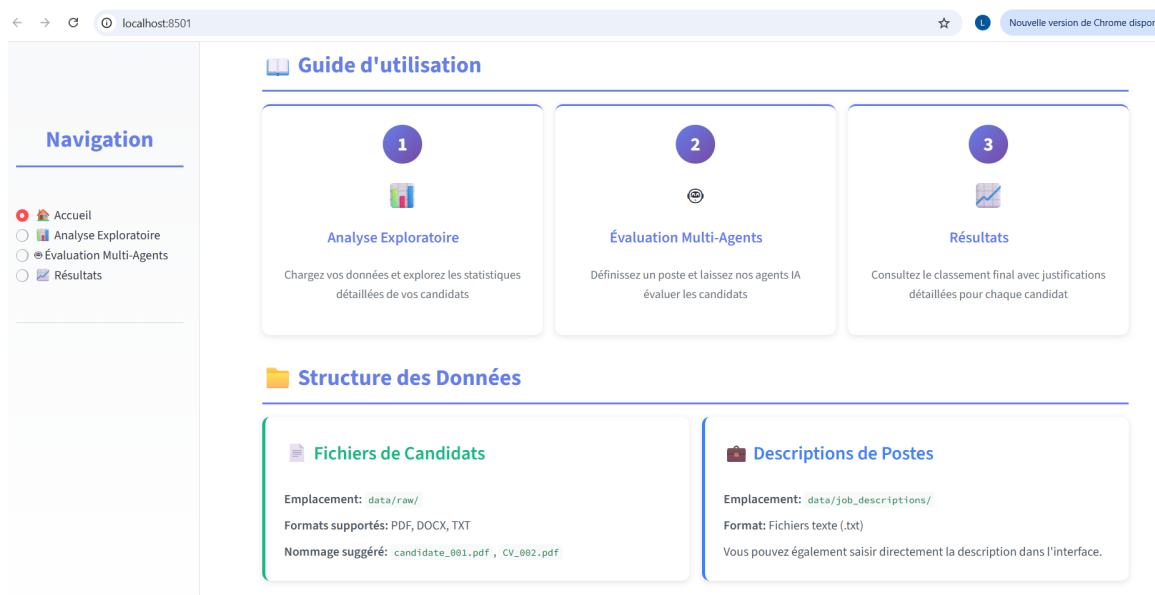


FIGURE 6 – Guide d'utilisation en 3 étapes



FIGURE 7 – Crédits développeuses

0.16.2 Page d'Analyse Exploratoire

Cette page permet au recruteur de charger et analyser les candidats avant l'évaluation. Elle affiche des statistiques détaillées et des visualisations interactives.

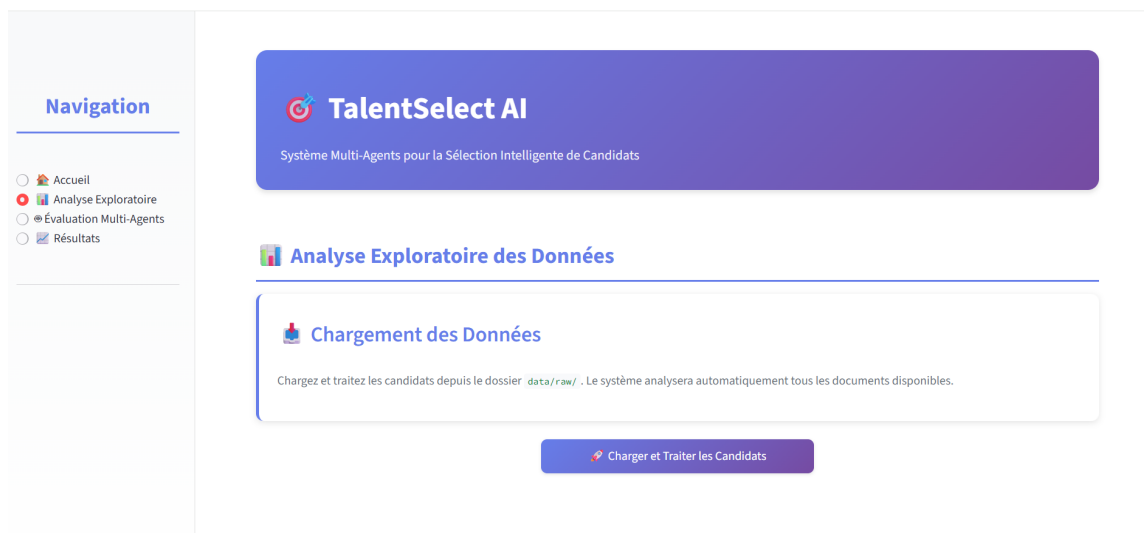


FIGURE 8 – Interface de chargement des candidats

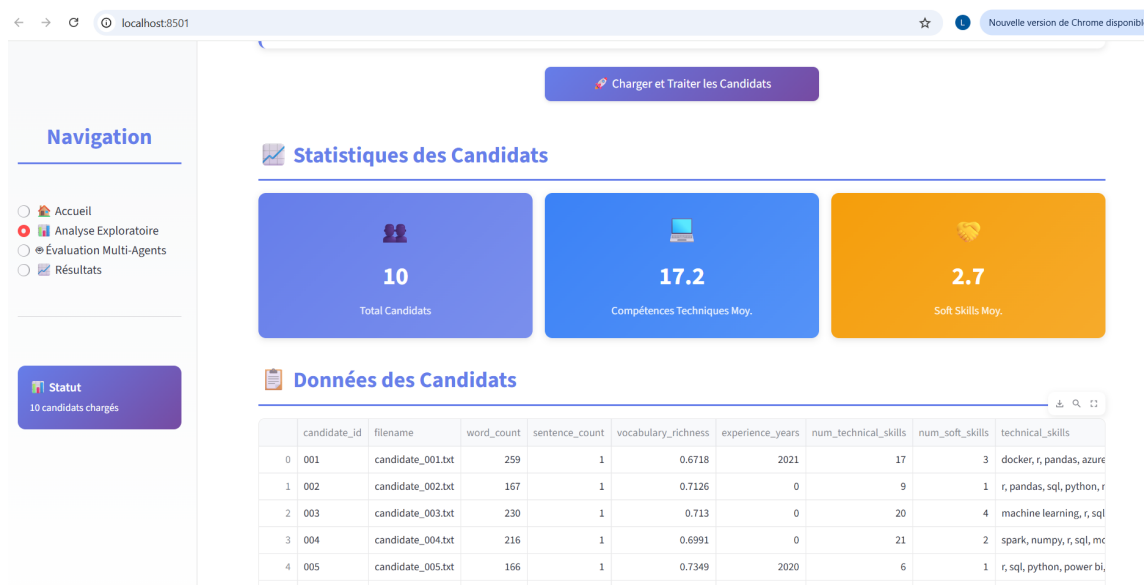


FIGURE 9 – Statistiques descriptives des candidats

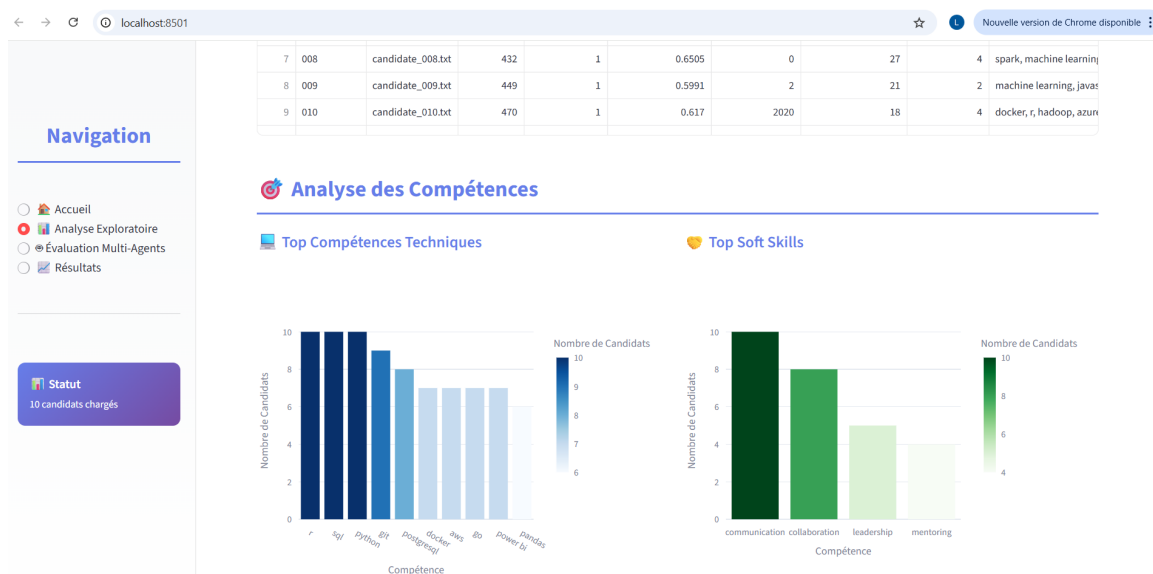


FIGURE 10 – Distribution des compétences techniques et soft skills

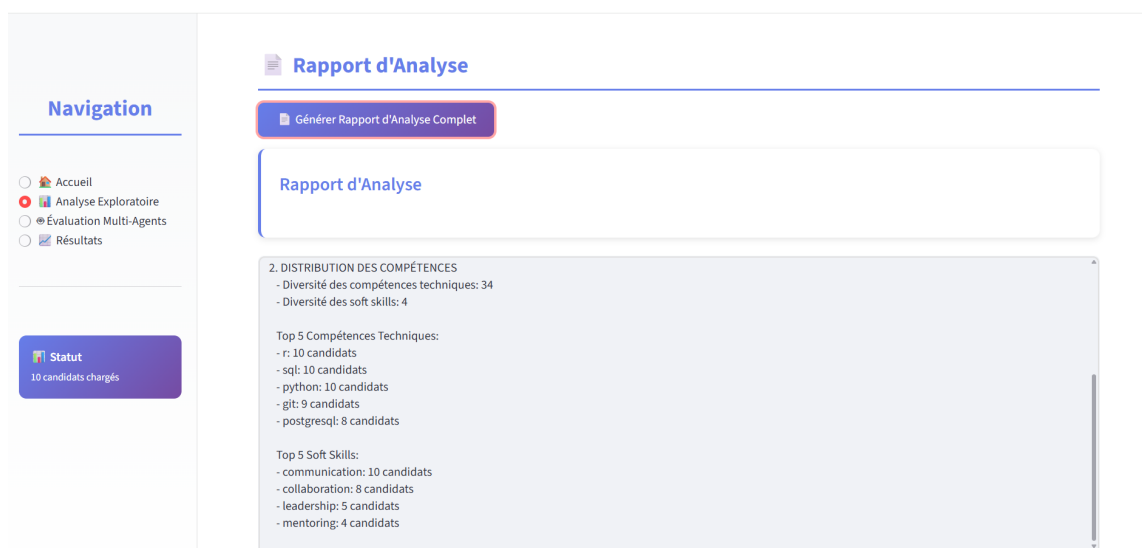


FIGURE 11 – option extraction du rapport

0.16.3 Page d'Évaluation Multi-Agents

Cette page permet de définir le poste à pourvoir et de lancer l'évaluation par les agents IA.



FIGURE 12 – Interface de saisie de la description de poste

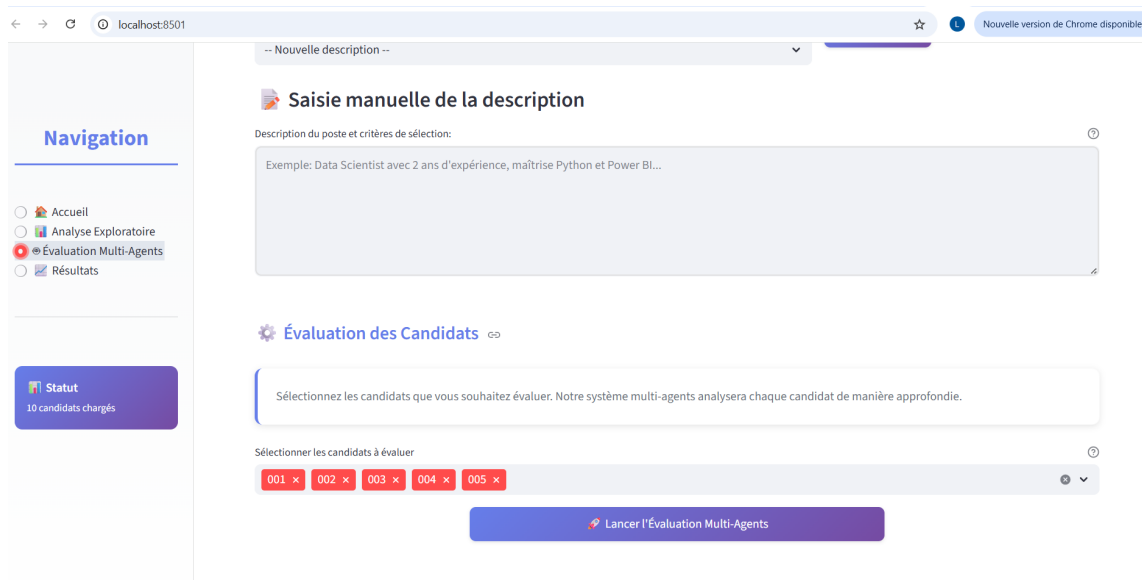


FIGURE 13 – Sélection des candidats à évaluer

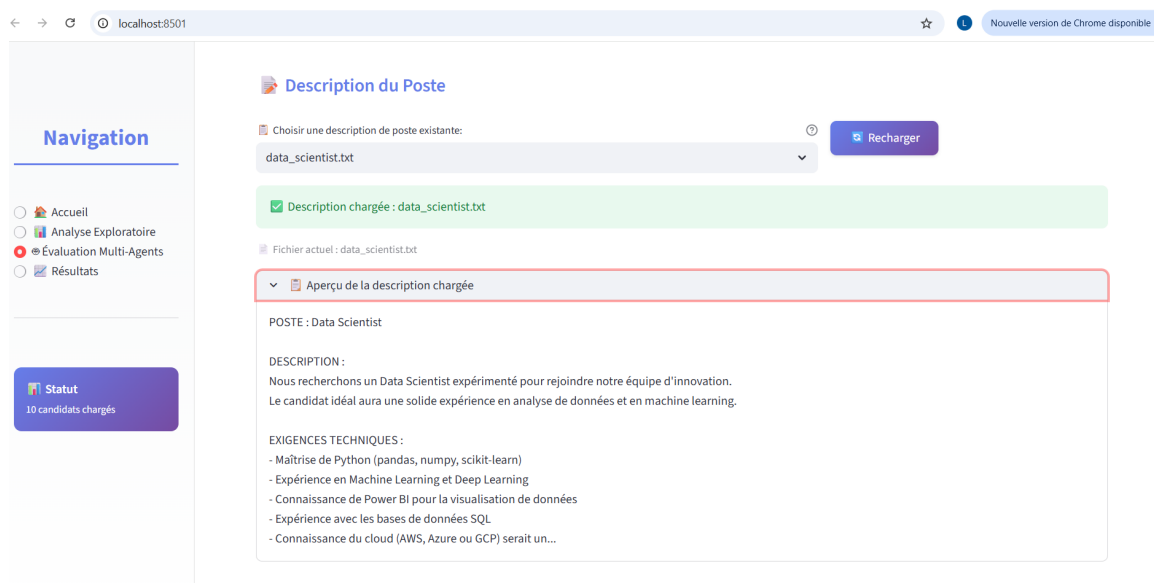


FIGURE 14 – Exemple de description de poste : Data Scientist

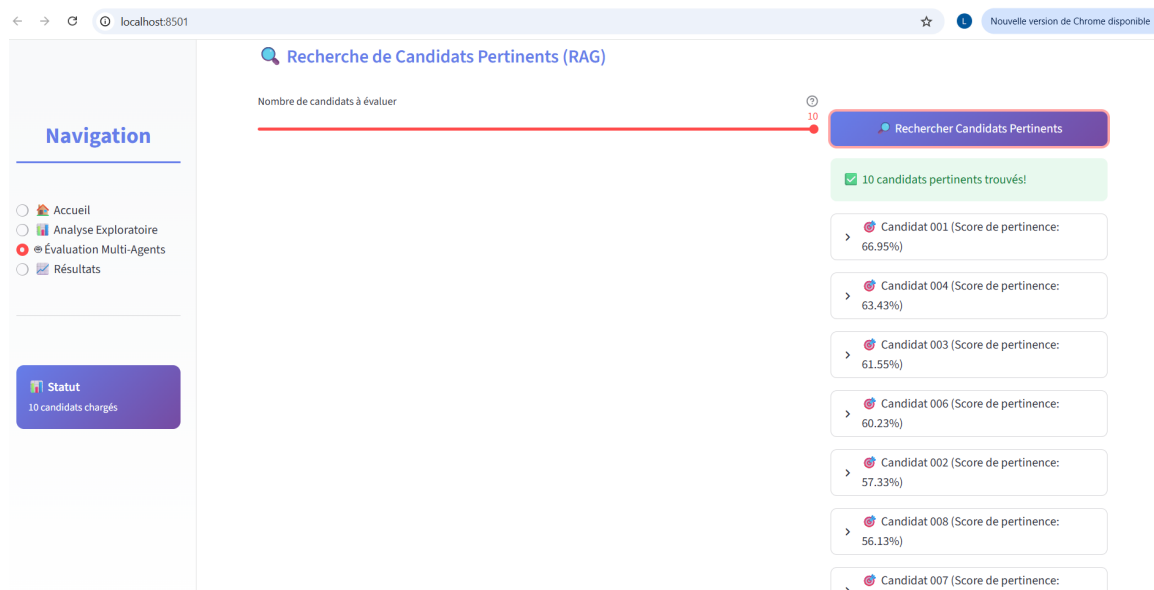


FIGURE 15 – Recherche RAG de candidats pertinents

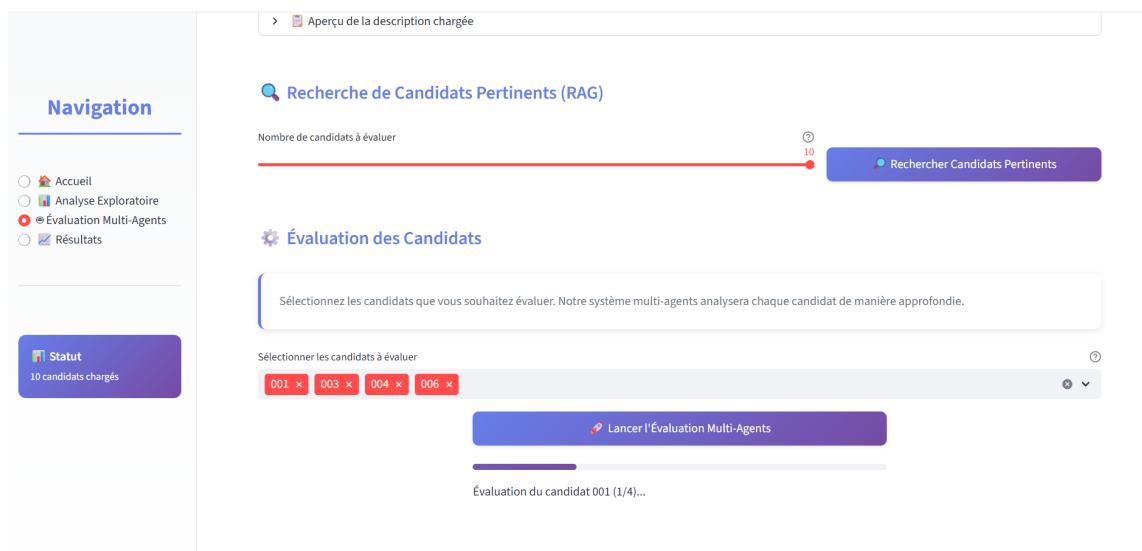


FIGURE 16 – Lancement de l'évaluation multi-agents

0.16.4 Page de Résultats

La page de résultats présente le classement final avec des justifications détaillées pour chaque candidat.

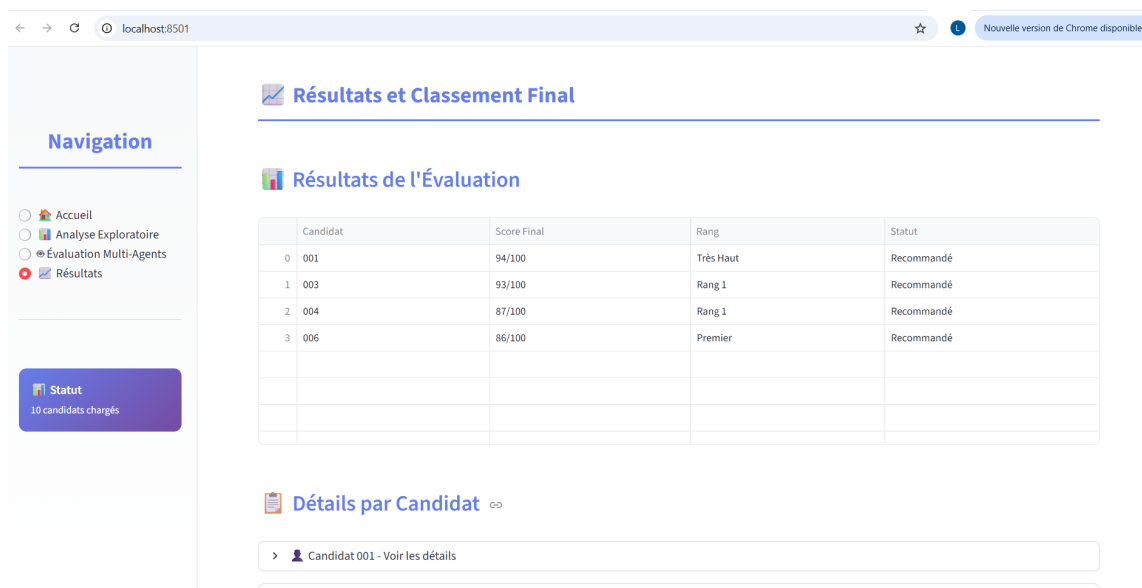


FIGURE 17 – Vue d'ensemble des résultats d'évaluation

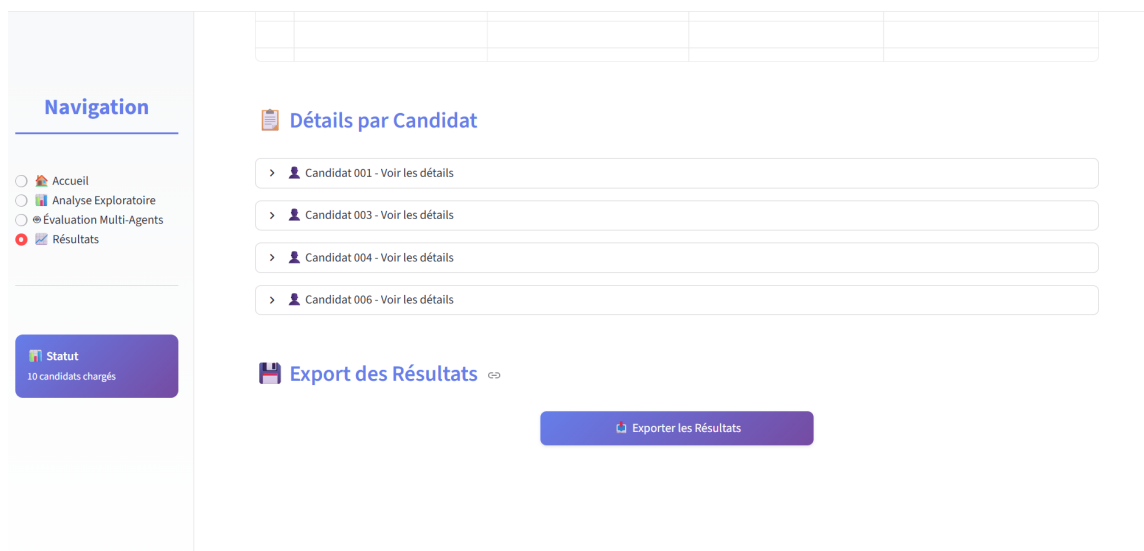


FIGURE 18 – Détails de l'évaluation d'un candidat

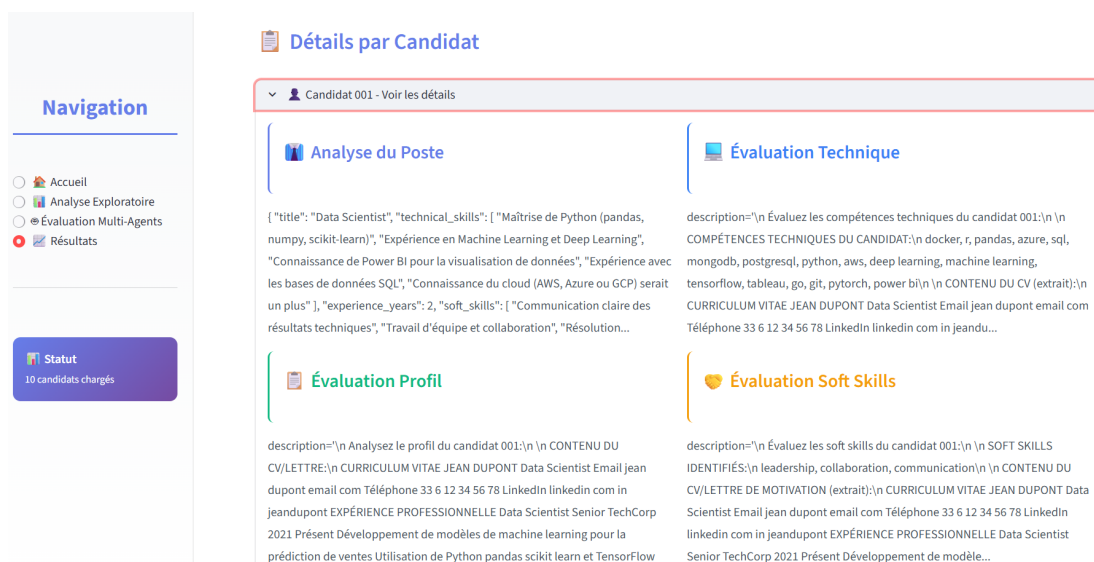


FIGURE 19 – Analyse détaillée par agent

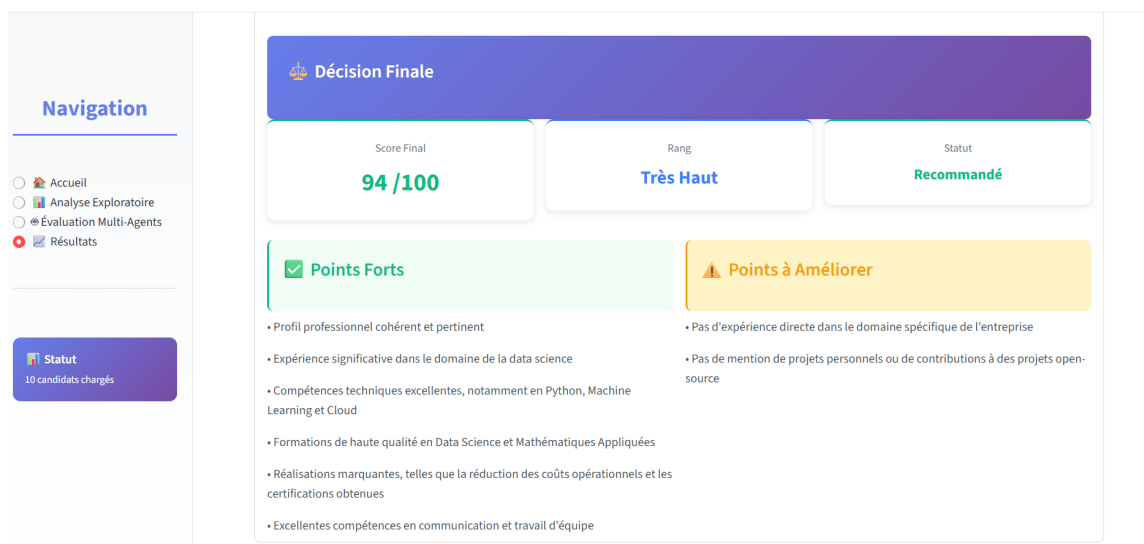


FIGURE 20 – Décision finale avec scores, justifications, points forts et points à améliorer

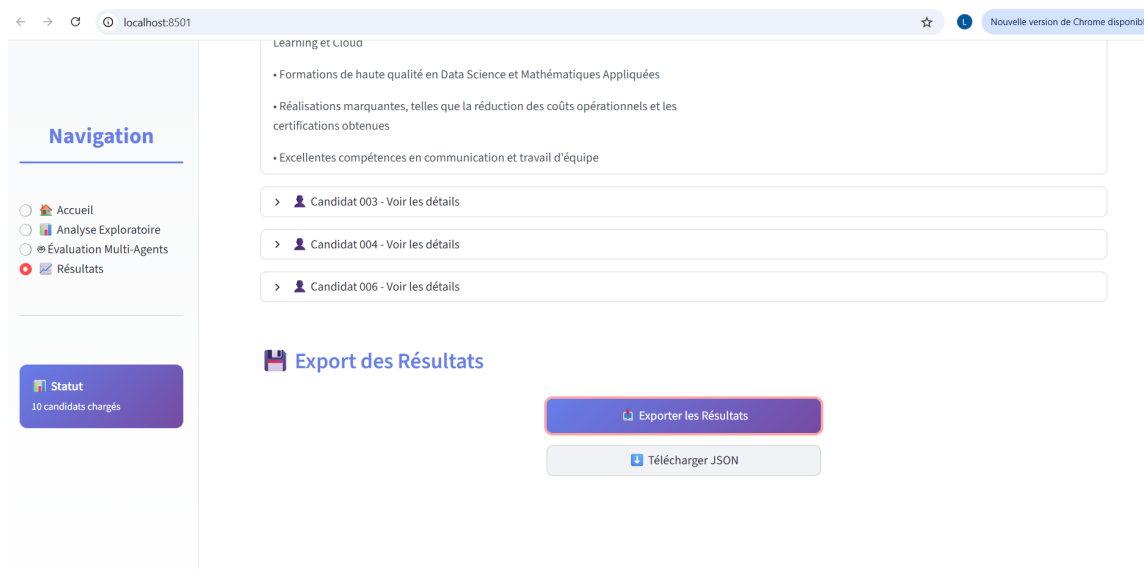


FIGURE 21 – Option export des résultats

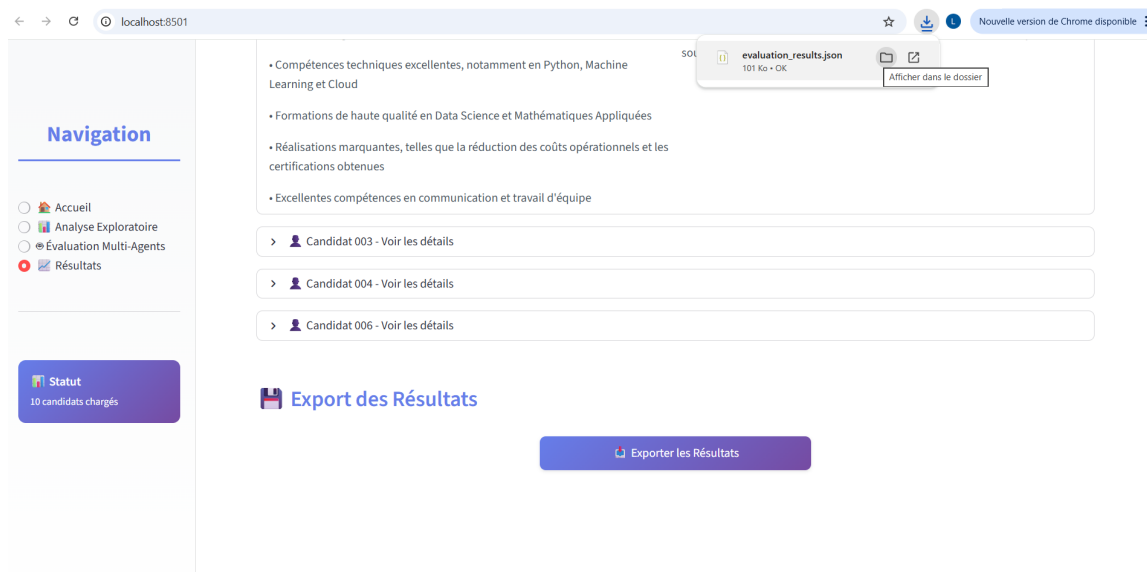


FIGURE 22 – Fichier json téléchargé

0.17 Démonstration d'un Cas d'Usage : Recrutement d'un Data Scientist

Pour illustrer le fonctionnement concret du système, un cas d'usage de recrutement pour un poste de Data Scientist a été implémenté.

0.17.1 Input

Les entrées fournies au système sont :

- **Description de poste** : "Data Scientist avec 2 ans d'expérience, maîtrise de Python et Power BI."
- **Corpus de candidats** : 5 CVs de candidats aux profils variés

0.17.2 Processus

Une fois le processus lancé depuis l'interface, le système exécute le workflow suivant en arrière-plan :

1. **Analyse exploratoire** : Les 5 CVs sont chargés, traités, et des statistiques initiales sont générées.
2. **Recherche RAG** : Le système identifie les candidats dont le profil sémantique correspond le mieux aux exigences de Python, Power BI et de l'expérience requise.
3. **Évaluation séquentielle** : L'équipe de 5 agents spécialisés est activée. Chacun analyse les candidats pertinents sous son propre angle :

- Agent RH : Extrait les critères du poste
- Agent Profil : Évalue la cohérence du parcours
- Agent Technique : Compare les compétences techniques
- Agent Soft Skills : Analyse les qualités interpersonnelles

4. **Agrégation et classement** : L'Agent Décideur collecte toutes les évaluations, calcule les scores finaux, génère le classement et rédige les justifications.

0.17.3 Output

Le résultat final est présenté à l'utilisateur dans la page de résultats. Il se compose d'un rapport synthétique affichant :

- **Classement final** : Top 3 des candidats avec leurs scores globaux (ex : Candidat A (92%), Candidat B (87%), Candidat C (84%))
- **Justifications détaillées** : Pour chaque classement, explication des forces et des faiblesses de chaque profil. Par exemple :
 - Candidat A : Forte expérience démontrée en Python et bon score sur Power BI
 - Candidat B : Profil technique très solide mais des faiblesses perçues en communication dans la lettre de motivation
- **Visualisations interactives** : Graphiques Plotly permettant d'explorer les scores par dimension (technique, soft skills, profil)
- **Export des résultats** : Possibilité de télécharger les résultats au format JSON pour intégration dans d'autres systèmes

Résultat du cas d'usage : Le système a réussi à identifier les 3 meilleurs candidats en moins de 2 minutes, avec des justifications détaillées pour chaque décision. Les scores étaient cohérents avec une évaluation manuelle effectuée en parallèle par un recruteur expérimenté.

Cinquième partie

Conclusion et Perspectives

Cette section finale récapitule les principales réalisations du projet et se tourne vers l'avenir. Elle synthétise les atouts fondamentaux de la solution développée et identifie les pistes d'amélioration et les travaux futurs.

0.18 Synthèse des Points Forts

Le système développé présente plusieurs avantages clés qui en font une solution robuste et pertinente pour moderniser le recrutement :

1. **Architecture Modulaire** : L'organisation claire et découplée du code garantit une excellente maintenabilité et facilite l'évolutivité future du système. Chaque module peut être amélioré indépendamment sans affecter les autres.
2. **Explicabilité Intégrée** : En fournissant des justifications détaillées pour chaque décision, le système va au-delà d'un simple classement et renforce la confiance de l'utilisateur dans les résultats produits. Cette transparence est essentielle pour l'adoption en entreprise.
3. **Recherche Sémantique (RAG)** : L'intégration d'un module RAG permet une pré-sélection très pertinente des candidats, en se basant sur la compréhension sémantique des besoins plutôt que sur une simple correspondance de mots-clés. Cela réduit significativement le nombre de candidats à évaluer manuellement.
4. **Interface Utilisateur Moderne** : Grâce à Streamlit et aux visualisations interactives, le système offre une expérience utilisateur intuitive et engageante, rendant la technologie accessible aux recruteurs non techniques.
5. **Extensibilité** : La conception du système facilite l'ajout de nouveaux agents ou de critères d'évaluation, permettant de l'adapter facilement à des besoins de recrutement spécifiques et variés.
6. **Performance** : L'utilisation de Groq API permet des temps de réponse très rapides (< 2 secondes par candidat), rendant le système utilisable en production.

0.19 Pistes d'Amélioration et Travaux Futurs

Bien que fonctionnel et performant, le système constitue une fondation sur laquelle de nombreuses améliorations peuvent être bâties :

0.19.1 Améliorations Techniques

- **Fine-tuning de modèles** : Entraîner ou affiner des modèles de langage spécialisés sur des corpus de données RH pour améliorer encore la pertinence des analyses et

des évaluations. Un modèle fine-tuné sur des milliers de CVs et descriptions de postes serait plus précis.

- **Intégration avec des APIs externes** : Connecter le système à des plateformes comme LinkedIn pour enrichir automatiquement les profils des candidats avec des données publiques (recommandations, certifications, etc.).
- **Amélioration du support multilingue** : Étendre les capacités du système pour analyser et évaluer des candidatures dans plusieurs langues (anglais, arabe, etc.) avec la même qualité d'analyse.
- **Optimisation de la recherche vectorielle** : Expérimenter avec d'autres modèles d'embeddings (sentence-BERT, multilingual models) pour améliorer la précision de la recherche RAG.

0.19.2 Améliorations Fonctionnelles

- **Mise en place d'une boucle de feedback** : Développer une interface permettant aux recruteurs de noter la pertinence des évaluations, afin de collecter des données pour l'amélioration continue du système via apprentissage actif.
- **Analyse de biais et promotion de l'équité** : Intégrer des modules dédiés à la détection et à la mitigation des biais potentiels dans les évaluations (biais de genre, origine, etc.) afin de promouvoir un processus de recrutement plus équitable.
- **Intégration avec des systèmes de suivi (ATS)** : Développer des connecteurs pour intégrer le système directement dans les flux de travail des systèmes de suivi des candidatures (Applicant Tracking Systems) existants comme Workday, Greenhouse, etc.
- **Tableau de bord analytique** : Créer un tableau de bord avancé avec des métriques de performance du recrutement (temps moyen de traitement, taux de correspondance, etc.).

0.19.3 Améliorations en Explicabilité

- **SHAP values pour les scores** : Implémenter l'analyse SHAP pour expliquer la contribution de chaque critère au score final de chaque candidat.
- **Visualisations d'explicabilité** : Créer des graphiques interactifs montrant l'importance relative de chaque dimension (technique, soft skills, profil) dans la décision finale.
- **Rapports d'audit** : Générer automatiquement des rapports d'audit détaillés pour la conformité réglementaire (RGPD, lois sur la non-discrimination).

Considérations éthiques : Toute amélioration future devra prendre en compte les aspects éthiques et légaux de l'IA en recrutement, notamment la transparence des algorithmes, la protection des données personnelles et la lutte contre les discriminations.

0.20 Conclusion

En conclusion, ce système multi-agents se positionne comme un atout stratégique, transformant le recrutement d'une fonction réactive en un processus proactif, piloté par les données et fondamentalement explicable. Il ne s'agit pas seulement d'un outil d'automatisation, mais d'une fondation pour une acquisition de talents plus intelligente et équitable.